Predictive Maintenance Pipeline

Engine + Alternator + Hydraulic Pump — Code & Logic Explained

A line-by-line walkthrough of every stage, function, constant, and design decision in the **pdm/** folder. Written so a reader who has the code open can understand not just *what* each piece does but *why* it had to be done that way, given what the raw Simulink data actually turned out to be.

Classes	Healthy, FlexibleShaft, PumpDisplacement, Leakage, GeneratorFault
Inputs	MATLAB Simulink sensor logs (.xlsx / .csv)
Model	XGBoost on scale-invariant + baseline-deviation features
Validation	GroupKFold by run (leakage-free)
GUI	Streamlit upload-and-predict app

1. The problem and the big idea

Three machines run as one coupled rig: a diesel engine drives a 3-phase Stamford alternator (feeding a rectifier and electrical load) and a Parker PV092 fixed-displacement hydraulic pump. We want predictive maintenance: feed a sensor log in and learn whether the rig is healthy or carrying one of four faults — flexible-shaft fault, pump-displacement fault, hydraulic leakage, or a (simplified) generator fault.

The pipeline is split into one shared engine and eight numbered stages, each stage being one notebook cell that reads the previous stage's cached artifact and writes its own. That makes every cell independently re-runnable and keeps all domain knowledge in exactly one place (pdm_common.py).

```
pdm_common.py        <- shared engine: config + all transforms + model factory
00_config.py         <- prints the configuration / data inventory
01_load_harmonize.py <- read raw logs -> harmonize -> resample -> clean (cache)
02_resample_clean.py <- inspect cleaned runs, show operating-point spread
03_label_window.py   <- time-aware labels + windowing (+ sequences for LSTM)
04_feature_extract.py<- per-window scale-invariant + deviation features
05_train_compare.py  <- GroupKFold model comparison (feature + sequence leagues)
06_evaluate_report.py<- final metrics, curves, SHAP, persist model.joblib
07_streamlit_app.py  <- upload-and-predict GUI (reuses the same transforms)
```

Why this design, in one sentence

The raw runs turned out to be recorded at wildly different operating points, so a naive classifier would learn the operating point instead of the fault. Everything else in this document is a consequence of defending against that.

2. What the raw data actually is (and why it dictates everything)

Before writing any model we profiled every file. The findings were not the clean uniform dataset one would hope for; each problem forced a concrete design choice.

2.1 Inconsistent capture windows

- Stop time was meant to be 0.3 s with the fault injected at 0.1 s.
- But the named-signal block of healthy data 1 and 2 only spans 0–0.002 s (2 ms of startup) with non-physical overshoot (speed 35,000 rad/s, 113 bar).
- X_Sevear flexible-shaft spans only 0–0.005 s: it never reaches the fault injection time at all, so it contains no fault.
- X_Critical spans 0–0.125 s: only ~0.025 s of fault region, too short for the minimum number of windows.

Consequence: those files are excluded with documented reasons (Section 3.4).

2.2 Incompatible operating points (the decisive finding)

Across runs the absolute signal levels differ by orders of magnitude:

run	pump pressure	V_dc
Healthy Data 3	1.013 bar (idle)	~28 V
disp1 (pump fault)	~1.5 bar	~182 V
Leakage_factor	~1900 bar	~144 V
leakage_fault(0.5)	~4126 bar	~116 V
generator fault	~47 bar	~183 V

A classifier trained on raw levels would simply separate runs by operating point, score a deceptively high accuracy, and detect nothing useful. This is exactly the trap the project brief warned about. The fix is to

make every feature invariant to operating point (Section 3.2).

2.3 Variable-step solver, non-uniform time

The logs come from a variable-step ODE solver, so the time column is non-uniform and the row counts range from ~8 k (xlsx) to ~1.05 M (the leakage CSVs, ~230 MB each). Near t = 0 the solver dumps a dense burst of femtosecond-scale steps full of numerical noise. We must resample onto a uniform grid and discard that init transient (Section 3.3).

2.4 Column-name chaos

Every file group names its columns differently. The named runs use labels like Pump_pressure:1; the flexible-shaft runs use generic scope names (PS-Simulink Converter16:1, etc.) and log a *different* set of channels. We identified the flex channels by matching their value ranges and time evolution against the named runs (Section 3.5). Only four channels are present in every run, so those four become the common modelling basis.

2.5 A duplicate run

Pump_Displacement.xlsx is byte-identical to disp1_fault(0.5).xlsx (max signal difference 0.0). Keeping both would put identical windows in train and test — hidden leakage — so the duplicate is dropped.

3. Core design decisions

3.1 Conservative labelling

Healthy windows come only from dedicated healthy runs. We do NOT relabel the pre-fault part of a fault run as healthy — pre-fault behaviour of a faulty machine is not a verified healthy operating condition. Fault windows come only from the fault-active region: t ≥ 0.1 s for timed faults, or the whole run for the always-on generator fault.

3.2 Operating-point invariance (the heart of it)

Every feature the model sees is one of two kinds, both dimensionless:

- Scale-invariant statistics — coefficient of variation (std/mean), crest factor, ripple, skew, kurtosis, zero-crossing rate, normalized slope, spectral centroid and low-band fraction, plus cross-signal correlations. Multiply a whole signal by 1000 and these do not change.
- Baseline-deviation — how a window's statistics differ from the same run's own pre-fault reference window [0.05, 0.10) s, divided by a robust scale. This self-normalizes each run, so a 28 V run and a 182 V run are compared by *how much they changed*, not by their absolute level.

Absolute raw stats are still computed but kept only for human-readable reporting, never fed to the model. A built-in self-check multiplies a test signal by 1000 and asserts the invariant feature stays identical (relative difference < 1e-6).

3.3 Resampling that erases the init transient for free

We interpolate every signal onto a uniform 10 kHz grid that starts at 0.05 s. Because the grid begins at 0.05 s, all the noisy near-zero solver steps are simply never sampled — no ad-hoc threshold needed. 10 kHz sits well above the electrical fundamental seen in the logs, decimates the MHz CSV logs and lightly up-samples the xlsx logs to one common rate so windows are comparable.

3.4 Documented exclusions

```
healthy data 1 / 2   -> 0-0.002 s startup garbage, non-physical overshoot
X_Sevear             -> 0-0.005 s: no fault-active region exists
X_Critical           -> fault region ~0.025 s: < MIN_WINDOWS windows
Pump_Displacement    -> byte-identical duplicate of disp1 -> leakage
```

3.5 Flexible-shaft channel identification (evidence, not guessing)

By sampling the generic flex columns at several times and matching their values to the named disp run at the same times, the four common channels were pinned down:

```
PS-Simulink Converter16 -> pressure (Pa)   (matches pump pressure baseline)
PS-Simulink Converter4  -> load current    (matches 28.7, 34.9, 35.0 ... A)
PS-Simulink Converter6  -> V_dc            (matches 144, 174, 175 ... V)
PS-Simulink Converter9  -> V_ac line       (oscillates +/-130..186 -> AC)
```

Speed, torque, flow and fuel are not identifiable in the flex scope set, so they are excluded from the shared model. The common basis is therefore {pressure, current, V_dc, V_ac} — confirmed automatically by stage 01 as the intersection of channels present in every run.

4. pdm_common.py — the shared engine

Imported by every stage and by the GUI, so training and serving run one identical code path (no train/serve skew). Walk through it top to bottom.

4.1 Paths (portable)

```
PDM_DIR = Path(__file__).resolve().parent
RAW_DIR = Path(os.environ.get('PDM_RAW_DIR', PDM_DIR.parent))
ART_DIR = PDM_DIR / 'artifacts'
```

Paths derive from the file's own location, so the folder runs on any machine. RAW_DIR (where the raw logs live) is only needed for training and can be overridden with the PDM_RAW_DIR environment variable.

4.2 Constants — every number justified

```
FAULT_T      = 0.10     # fault injection time, confirmed from the flag columns
STOP_T       = 0.30     # nominal stop time
BASELINE_WIN = (0.05, 0.10)  # per-run pre-fault reference; also grid start
FS           = 10_000   # uniform resample rate (Hz), above the electrical fund.
WINDOW_SEC   = 0.02     # 200 samples/window: stable stats, several windows/run
STRIDE_FRAC  = 0.5      # 50% overlap: limits correlation between windows
MIN_WINDOWS  = 3        # a run must yield >=3 fault windows to be included
```

None of these are arbitrary: FAULT_T comes from the data, BASELINE_WIN is the pre-fault slice, FS from the Nyquist argument, WINDOW_SEC trades stable statistics against window count, STRIDE_FRAC controls overlap leakage, MIN_WINDOWS is the include/exclude gate.

4.3 RATED references and unit conversions

RATED holds the datasheet operating points (torque 500 N·m, 2200 rpm, 28 V dc, 535 V ac, 200 bar). Because the 0.3 s sims rarely reach these, RATED is used only as a physics reference for percentage-deviation features, never as the learned healthy distribution. Unit helpers convert raw Simulink units to model units: Pa→bar (/1e5), rad/s→rpm (×60/2π), m³/s→L/min (×6e4).

4.4 FILE_MAP and EXCLUDED

FILE_MAP maps each included raw file to its class label, an always_on flag, and a dictionary translating canonical names to that file's raw column names. The named runs share one scheme; the flex runs use the four-channel scheme from Section 3.5. EXCLUDED lists every dropped file with its reason, so the report can state exactly what was left out and why.

4.5 read_run + harmonize

read_run loads csv/xlsx, keeping only the time column plus the mapped raw columns; the duplicate time/Constant/scope columns are ignored by selecting columns by exact name. harmonize renames raw→canonical, applies the unit conversion, coerces to numeric, drops bad times, and sorts by time.

4.6 resample_uniform

```
t0   = max(BASELINE_WIN[0], t.min())     # start at 0.05 s -> drops init transient
grid = np.arange(t0, t.max(), 1/FS)      # uniform 10 kHz grid
out[c] = np.interp(grid, t, v)           # linear interpolation per signal
```

Linear interpolation onto the uniform grid. Starting at 0.05 s is what discards the solver init noise without any magic threshold.

4.7 hampel_clip + clean_run

hampel_clip is a Hampel filter: for each point, compute the rolling median and the rolling median-absolute-deviation; replace points more than 3 robust sigma (1.4826×MAD) from the median. This removes solver spikes without distorting genuine dynamics, because it is median-based. clean_run applies it to every signal and leaves constant signals untouched.

4.8 baseline_stats

For each signal it computes mean, std and rms over the pre-fault window [0.05, 0.10), plus a robust scale = IQR, falling back to std, then |mean|, then 1. That scale makes the deviation features dimensionless and division-safe.

4.9 iter_windows

```
lo   = 0.05 if always_on else FAULT_T    # fault-active region start
wlen = int(WINDOW_SEC*FS)                 # 200 samples
step = int(wlen*STRIDE_FRAC)              # 100 samples (50% overlap)
for start in range(i0, i1-wlen+1, step):  # only COMPLETE windows
```

Yields complete windows over the fault-active region. Timed faults start at 0.1 s; the always-on generator fault uses the whole run. Healthy runs use the same t ≥ 0.1 s region so healthy and fault windows occupy the same time region — that removes any chance of the model keying on absolute time.

4.10 window_features — what the model actually sees

For each of the four common signals the function computes the invariant and deviation features below; all are dimensionless.

```
cov     = std/|mean|                ripple  = (max-min)/|mean|
crest   = max|x|/rms                skew, kurtosis
zcr     = zero-crossing rate        slope   = polyfit slope * n / |mean|
speccen = spectral centroid (Hz)    speclow = low-band energy fraction
dmean   = (mean - base_mean)/scale  dstd = (std-base_std)/scale
drms    = (rms  - base_rms )/scale
cross-signal: corr(V_dc, I), corr(pressure, V_ac), CoV of the V_dc*I power proxy
```

That is ~12 features per signal plus three cross-signal terms = 51 features total. The spectral features are deliberately treated as coarse shape descriptors only: a 0.02 s window gives ~50 Hz frequency resolution, so they cannot resolve fine low-frequency content and we do not lean on them.

4.11 build_run_features, process_file, process_uploaded, infer_spec

build_run_features ties it together for one run: compute the baseline, iterate windows, extract features, and return nothing if the run yields fewer than MIN_WINDOWS windows. process_file is the training path (raw file → clean → features). process_uploaded is the GUI path: infer_spec picks the FILE_MAP scheme whose raw columns best match the uploaded file, then the same harmonize→clean→features chain runs. One code path for train and serve.

4.12 _XGBSafe and build_feature_models

_XGBSafe wraps XGBoost and label-encodes y internally. This is needed because GroupKFold, with single-run classes, can hand a fold a non-contiguous subset of class labels, which raw XGBoost rejects; the wrapper keeps XGBoost usable. build_feature_models returns the four feature-league pipelines, each with feature selection as an in-pipeline step (so it is fit per fold) and scaling only where it matters:

```
RandomForest -> SelectKBest -> RF            (no scaler: trees are scale-free)
ExtraTrees   -> SelectKBest -> ET            (no scaler)
XGBoost      -> SelectKBest -> XGBSafe       (no scaler)
SVM_RBF      -> StandardScaler -> SelectKBest -> SVC   (SVM needs scaling)
```

5. The pipeline stages

Stage 00 — config

Prints the constants, the included runs, the excluded files with reasons, the per-class run counts, and the expected common signals. Pure documentation so the notebook records exactly what fed the model.

Stage 01 — load + harmonize + resample + clean

The only heavy-IO stage. The 230 MB CSVs are read once here; ingest, harmonize, uniform-resample and outlier-clean happen together and the small cleaned result (~2.5 k rows/run) is cached to clean_runs.parquet. It also writes run_availability.csv and derives the common signals as the intersection of channels present in every run — which comes out as exactly {pressure, current, V_dc, V_ac}, matching the documented expectation.

Stage 02 — inspect cleaned runs

Loads the cache and prints the per-run operating point of the common signals in the eval region. This is where the order-of-magnitude spread (pressure 0.03–4126 bar) is shown explicitly — the evidence that justifies the invariant-feature design. It also plots V_dc per run.

Stage 03 — labelling + windowing

Applies the conservative labels, windows each run inside its fault-active region, reports the window count per run, and drops runs below MIN_WINDOWS. It also saves the raw windowed sequences (shape n×200×4) to windows.npz for the LSTM league, and prints the class balance and runs-per-class — including the blunt note that Healthy and GeneratorFault have only one run each.

Stage 04 — feature extraction

Builds the full 170×51 feature matrix (no selection here — selection is done inside the CV folds to avoid leakage) and saves features.parquet. It prints per-class means of a few features as a sanity check that classes differ by fault, not by scale.

Stage 05 — model comparison

The methodological core. Key choices, each defending against a specific error:

- GroupKFold by run_id — windows of one run never split across train and test. An assertion checks train/test runs are disjoint in every fold.
- Selection inside the pipeline — SelectKBest is a pipeline step, so it is fit only on each fold's training split. No global pre-selection, no selection leakage.
- Scaling only for SVM — trees get none.
- Two leagues, never cross-ranked — the feature league (RF / ExtraTrees / XGBoost / SVM on engineered features) and the sequence league (LSTM on raw windows). It is not fair to call an LSTM worse than a tree when they receive different inputs, so they are reported separately.
- Selection metric = macro-F1, with balanced accuracy and minimum per-class recall also reported. Plain accuracy is shown but not used to choose, because the single-run classes make it misleading.
- Minimal tuning — informed defaults only, no big hyper-parameter search, because with ~10 runs a search would overfit itself.

It selects the best feature-league model by macro-F1 (XGBoost) and writes model_comparison.csv. If TensorFlow is unavailable the LSTM is skipped with a logged note and the feature league stands as the deployment basis.

Stage 06 — final evaluation and persistence

Produces a leakage-free out-of-fold report by looping the GroupKFold folds manually, mapping each fold's probability columns back into the global class space (needed because some folds lack a class). It reports accuracy, balanced accuracy, macro and per-class F1/recall, the confusion matrix, ROC (one-vs-rest), precision-recall and calibration curves, and per-class distribution stats (mean, median, skewness, kurtosis). Calibration matters because the GUI shows a confidence number. SHAP gives global feature importance (falling back to built-in importances if SHAP is unavailable). Finally the chosen pipeline is refit on all data and saved as model.joblib, with meta.json recording the feature list, window parameters and class names.

Stage 07 — Streamlit GUI

Loads model.joblib and meta.json, takes an uploaded xlsx/csv, runs the exact same pdm_common transforms, predicts each window, and aggregates to a verdict (Healthy or which fault) with a mean-probability confidence, a per-window vote table, a probability bar chart, and a plot of the harmonized signals.

6. Results and honest limitations

Best model: XGBoost (feature league). Leakage-free out-of-fold scores:

Class	F1	Status
Leakage	0.99	3 runs - validated
FlexibleShaft	0.85	2 runs - validated
PumpDisplacement	0.83	3 runs - validated
GeneratorFault	0.00	1 run - cannot validate
Healthy	0.00	1 run - cannot validate

Overall accuracy 0.79, balanced accuracy 0.59. The three multi-run fault classes are genuinely well separated and properly validated. The two single-run classes score zero under grouped cross-validation for an unavoidable reason: when their only run is the test fold, the model never saw that class in training, so it cannot predict it. This is the honest worst case, not a bug. The deployment model is refit on all data, so at serve time it does predict all five classes (verified: each known file is classified correctly).

The single most valuable improvement would be to regenerate Healthy and GeneratorFault with at least three runs each, ideally at operating points that overlap the fault runs. That alone would let those classes be validated and would lift balanced accuracy substantially.

- LSTM comparison is skipped unless TensorFlow is installed (no Python-3.14 wheel at build time).
- The flexible-shaft channel mapping is evidence-based but a calibration knob: if it is ever found wrong, only FILE_MAP needs editing.

7. Running it

```
# GUI only (model already trained):
pip install -r requirements.txt
streamlit run 07_streamlit_app.py

# Retrain from raw data:
#   keep pdm/ inside the folder holding the raw logs, OR
#   set PDM_RAW_DIR to that folder, then run the notebook cells 01-06.
```

That is the whole pipeline: profile the data honestly, neutralize the operating-point confound with invariant and baseline-deviation features, validate without leakage, and report what can and cannot be trusted.